

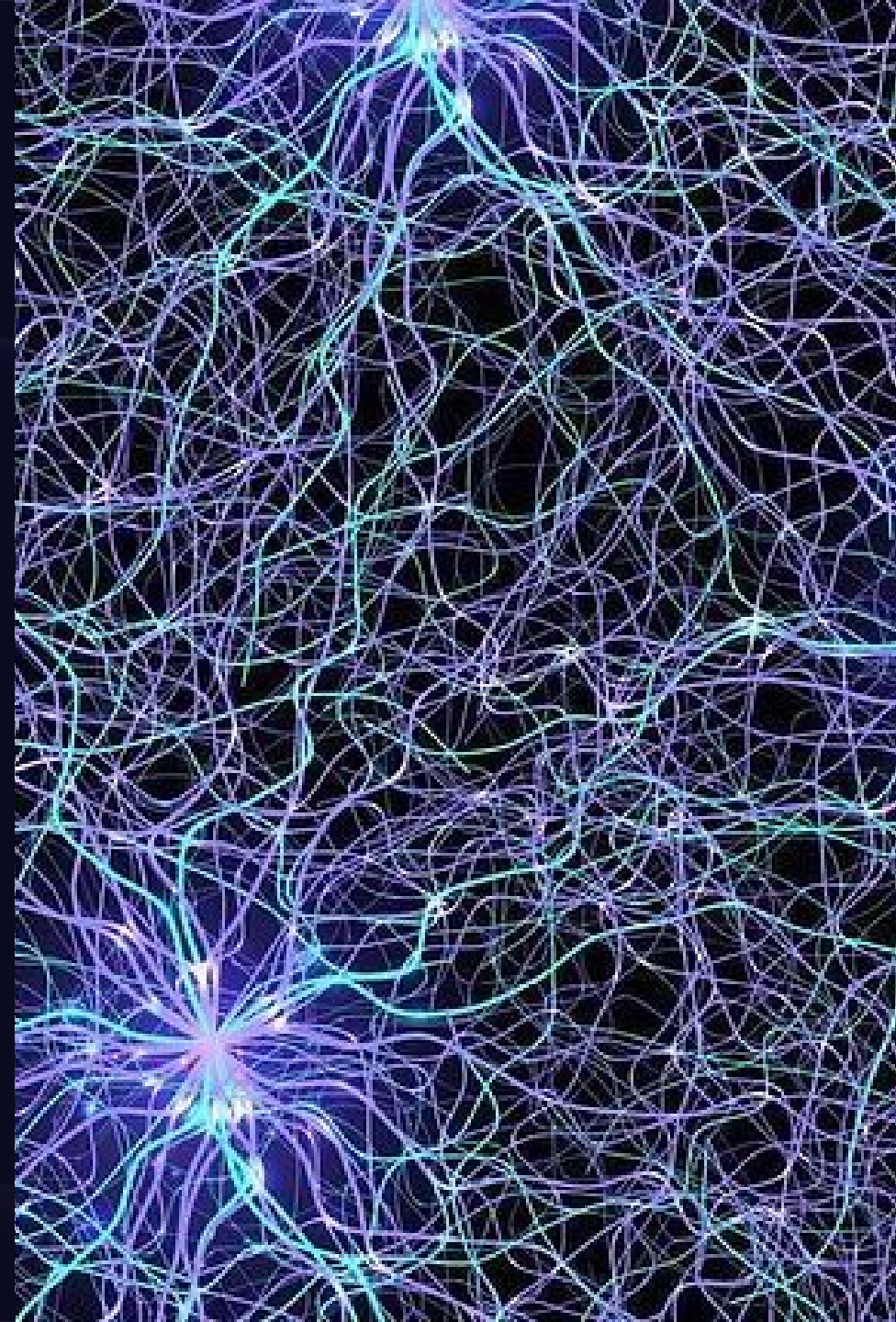
From Generative AI to Agentic AI

Building Enterprise AI Agents with Python

Ramesh Sannareddy

2-DAY AGENTIC AI WITH PYTHON

CLAUDE / AZURE OPENAI · PYTHON · ENTERPRISE WORKFLOWS



What is an AI Agent?

Before we dive in — let's test your intuition. Each of these terms gets used interchangeably. By the end of this session, you'll be able to tell the difference precisely — and you'll have built one in Python.

Chatbot

Responds to messages in a conversational interface

LLM Application

Wraps an LLM with a prompt and returns a response

AI Assistant

Interacts conversationally, may recall context

AI Agent

Pursues goals, uses tools, reasons, and adapts



"By the end of these two days, you'll be able to tell the difference — and you'll have built one."

Why Are We Here?

Traditional enterprise software follows a rigid path: a user triggers an application, business logic fires, a database or API is called, and a result is returned. Generative AI shortened that to a user asking an LLM and getting an answer. But neither model can *accomplish* something on your behalf.

What We Can Do Today

"Explain our employee onboarding process."

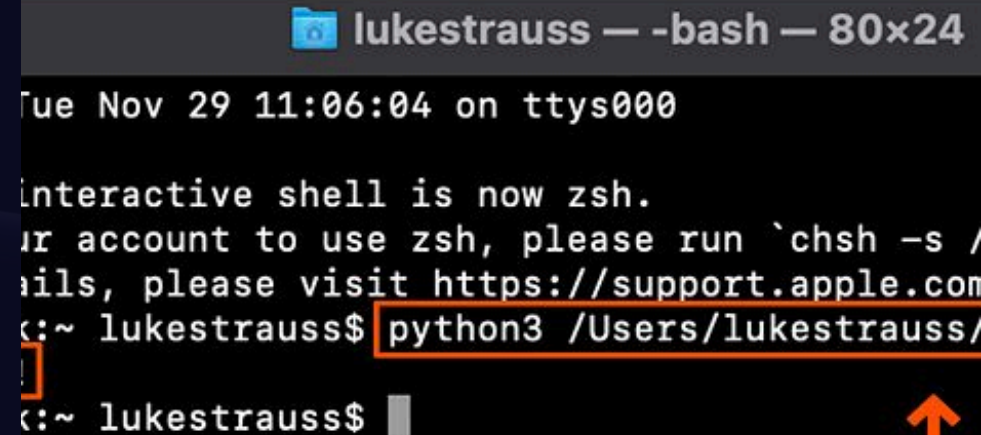
"What is our expense policy?"

What We Want to Do

"Onboard Priya."

"Review this expense, check the policy, identify exceptions, and route it for approval."

This training is explicitly designed for **internal enterprise workflow automation** — not personal productivity tools or generic chatbot scenarios.



A terminal window titled 'lukestrauss — -bash — 80x24'. The window shows a session starting with 'Tue Nov 29 11:06:04 on ttys000'. It then displays 'Interactive shell is now zsh.' followed by a message: 'For your account to use zsh, please run `chsh -s /bin/zsh`. For more details, please visit <https://support.apple.com>'. The prompt is 'lukestrauss\$'. The command 'python3 /Users/lukestrauss/' is entered and highlighted with a red box. The prompt returns to 'lukestrauss\$'.

```
lukestrauss — -bash — 80x24
Tue Nov 29 11:06:04 on ttys000

Interactive shell is now zsh.
For your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com
lukestrauss$ python3 /Users/lukestrauss/
lukestrauss$
```



Input

The Evolution of AI

Every era of AI expanded what machines could do. The critical transition for this course is the shift from *generating an answer* to *accomplishing a goal*.

1

Traditional AI

Rules and deterministic decisions

2

Machine Learning

Learn patterns from data

3

Generative AI

Generate content on demand

4

AI Assistants

Interact conversationally

5

Agentic AI

Execute multi-step workflows autonomously

📝 Discussion question: "If I give an LLM access to your company's APIs, is that automatically an agent?" — hold this question. We'll return to it.

Generative AI Is Not Automatically Agentic AI

Access to an LLM does not make a system agentic. The difference lies in what happens after the prompt is sent. Generation produces content. Agentic systems pursue outcomes.

Generative AI



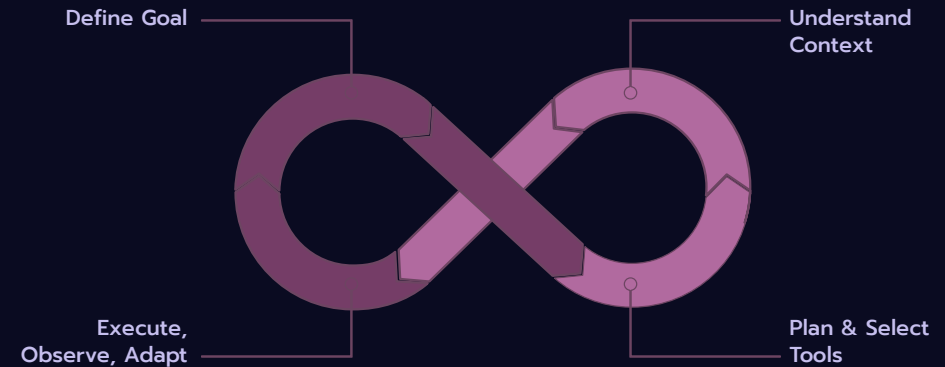
Prompt

LLM

Response

Produces content in response to a prompt

Agentic AI



Pursues outcomes: "Onboard employee EMP-1042."

What Do We Mean by an AI Agent?

An AI agent is an LLM-powered software system that can pursue a goal by reasoning about the task, deciding what actions are needed, using available tools, observing results, and continuing until the task is completed or requires human intervention.



Goal



Reason



Decide



Act



Observe



Adapt



Human Intervention

This definition maps directly onto the course's core themes: autonomy, planning, goal-oriented execution, memory, reasoning, tool usage, reflection, and human-in-the-loop design.

The Anatomy of an Agent

An agent is not a single technology — it is a set of cooperating capabilities. Each one below will be progressively implemented in Python across the labs.



Goal + Instructions

What the agent is trying to achieve and the behavioral constraints it must follow



Context + Memory

What the agent currently knows — short-term working state and longer-term persistent memory



LLM / Reasoning

The intelligence layer that interprets context, evaluates options, and decides what to do next



Planning + Tools + Action

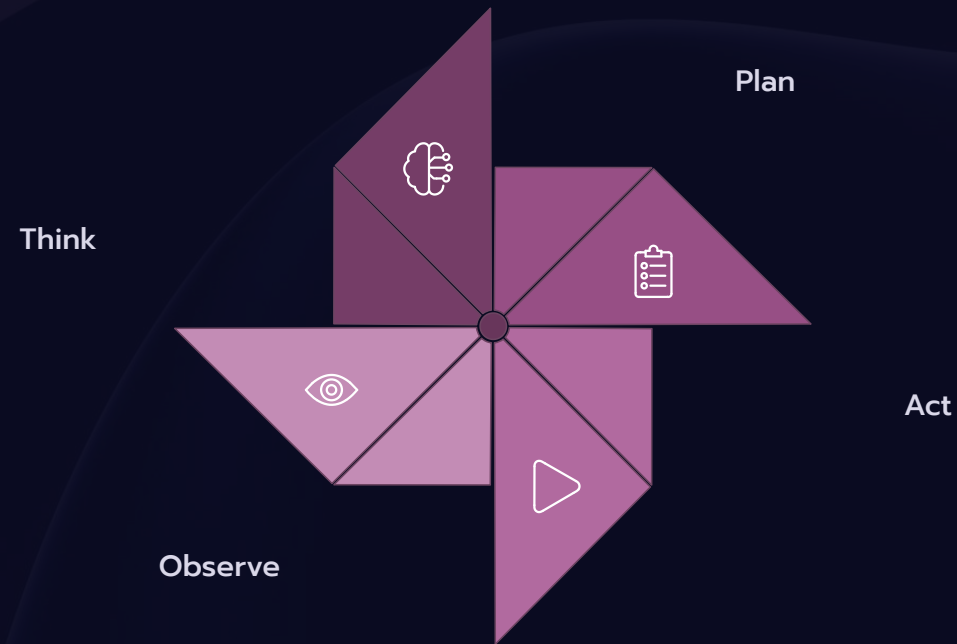
Deciding which tools to invoke, executing actions against enterprise systems, and feeding observations back into the loop



"These aren't eight separate technologies. They are capabilities we will progressively implement in Python."

The Agent Loop

Think → Plan → Act → Observe → Repeat



Example Goal

"Onboard a new software engineer."

Agent Steps

1. Identify employee and validate record
2. Determine role and required access
3. Check equipment availability
4. Schedule induction session
5. Generate checklist and welcome communication

The loop is the heart of this course. Every lab builds toward a deeper implementation of this Think → Plan → Act → Observe → Repeat model.

n Oversight, Enterprise S



Autonomy: What Does It Actually Mean?

Autonomy does **not** mean "let the LLM do whatever it wants." In enterprise contexts, autonomy is always bounded — defined by the policies, tools, and approval gates you explicitly design into the system.

Level 1 — Assist

Agent suggests what to do. A human reviews and acts.

Level 2 — Execute with Approval

Agent proposes actions → human approves → agent executes. Full auditability.

Level 3 — Bounded Autonomy

Agent executes predefined actions within strict policies and guardrails without per-action approval.

This framework prepares you directly for **Lab 3's human-approval and guardrail architecture**, where you will implement approval gates and constraint validation in Python.

Why Enterprise Agents Need Tools

An LLM knows an enormous amount — but knowledge alone cannot change the state of an enterprise system. Without tools, the agent can only talk about work. With tools, it can do work.

What an LLM Cannot Do Alone

- Query your HR or ITSM system
- Reserve a laptop from inventory
- Inspect a live database record
- Schedule a meeting on a calendar
- Update an enterprise application
- Trigger an approval workflow

Tools Bridge the Gap

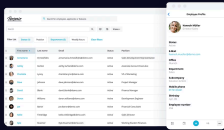
By exposing APIs as callable tools, the LLM can instruct the agent to interact with HR systems, ITSM platforms, databases, calendars, and any enterprise integration layer.

Tools turn an LLM from a conversational system into a system that can interact with your enterprise.

This sets up **Lab 2** directly.

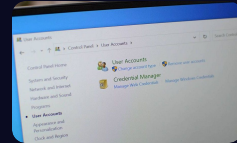
Example: Employee Onboarding Agent

This is the central running example for the entire course. A single natural-language command triggers a 12-step agentic workflow that touches multiple enterprise systems.



1–3: Identify & Validate

Retrieve EMP-1042 from the employee directory, validate the record, and determine role and department.



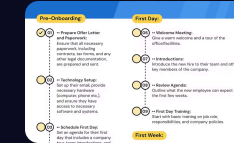
4–6: Provision Resources

Determine required system access, check equipment availability, and find an induction calendar slot.



7–9: Plan & Approve

Build the onboarding plan, surface it for human approval, and execute only the approved actions.



10–12: Deliver & Report

Generate the onboarding checklist, draft the welcome email, and report completion to the requester.

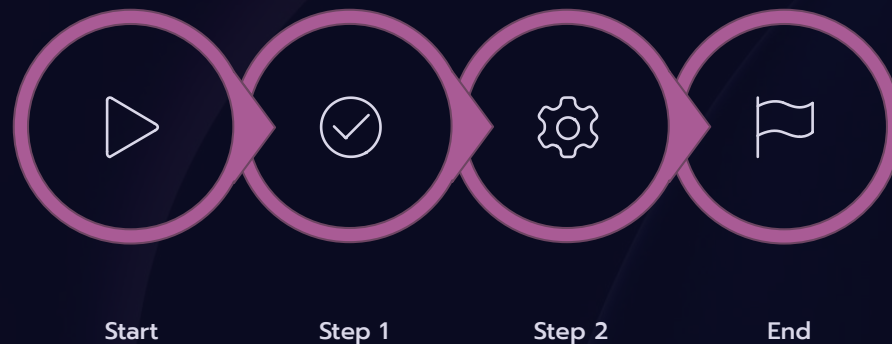


This scenario becomes the major workflow in **Lab 3**. Keep it in mind as our reference anchor throughout the two days.

Agent vs. Conventional Workflow

This is a critical architectural distinction for senior engineers. Conventional workflows are explicit state machines — every branch is pre-coded. Agentic workflows are goal-directed — the agent determines the next action based on current state.

Conventional Workflow



"Follow these steps." Logic is designed upfront. Every decision branch must be explicitly coded.

Agentic Workflow



"Given the goal and current state, determine the next appropriate action." The path emerges at runtime.

⚠ Do not imply agents should replace conventional workflows. The course explicitly addresses when **not** to use agents and the design trade-offs involved.

Where the Intelligence Actually Comes From

An agent is not magic. Its behavior is entirely the product of five cooperating ingredients. For architects in the room: **the LLM is one component of an agent — not the entire agent.**

1

LLM

Reasoning, language understanding, and decision-making

2

Instructions

Behavioral constraints, persona, and guardrails that bound what the agent will do

3

Tools

Executable capabilities that connect the agent to enterprise systems and data

4

State / Context

What the agent knows right now — working memory, retrieved data, prior observations

5

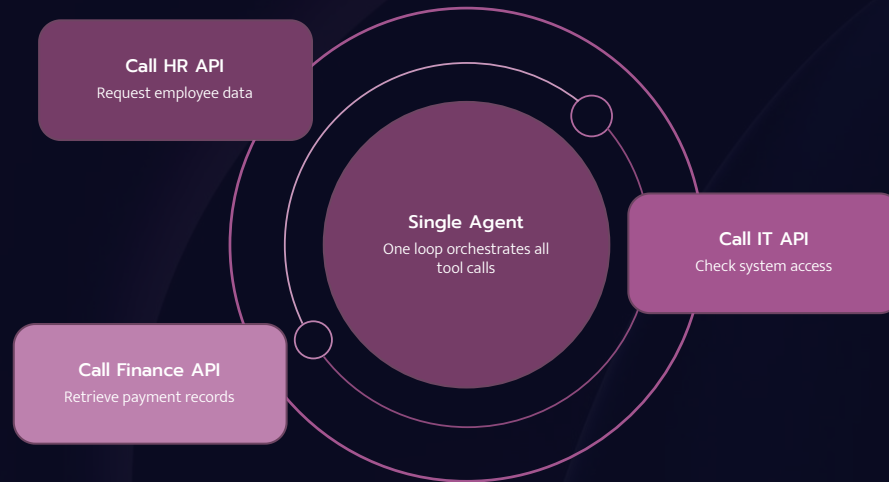
Control Logic

Policies, validation rules, and approval gates that govern what the agent is allowed to do

Single Agent vs. Multi-Agent

Multi-agent architecture introduces real coordination complexity. The guiding principle: don't reach for it unless a single agent genuinely cannot do the job cleanly.

Single Agent



One reasoning loop, one context, simpler to debug and audit. Covers most enterprise workflows in this course.

Multi-Agent



Coordinator delegates to specialist agents. Adds parallelism and separation of concerns — but also failure surfaces and state synchronization complexity.

Day 2 will cover Router, Planner, Executor, Reviewer, Coordinator patterns, state machines, and escalation. **Don't introduce multi-agent just because you can.**

Common Enterprise Agent Use Cases

These domains represent the capstone scenarios in the course syllabus. Each follows the same fundamental pattern: a business goal, access to enterprise data and tools, a reasoning agent, and a human approval gate before consequential actions are taken.

IT & HR Operations

Helpdesk automation, employee leave assistant, and full employee onboarding with access provisioning

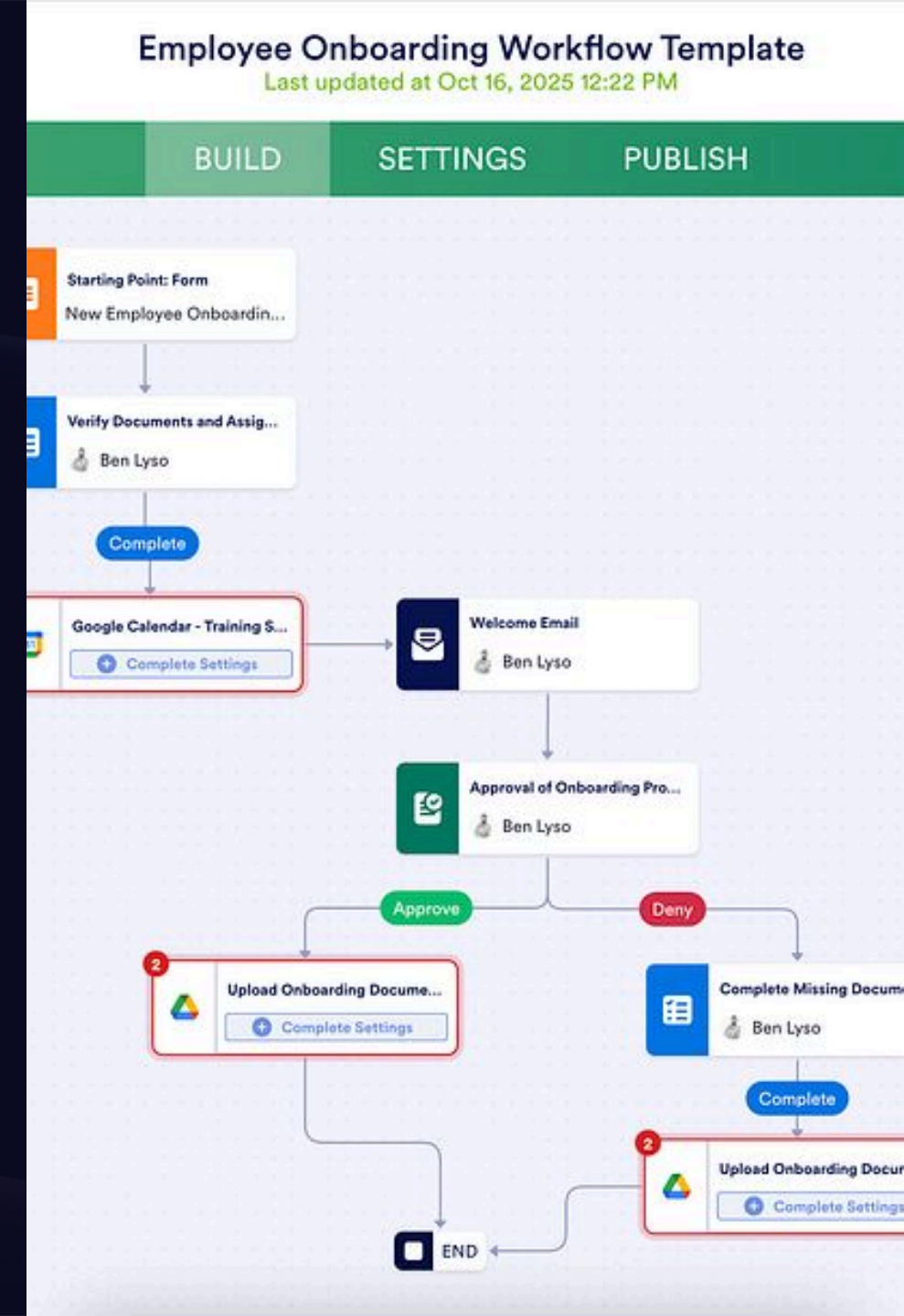
Procurement & Finance

Purchase approval workflows, expense processing with policy validation, and vendor onboarding

Legal & Knowledge

Contract review with clause extraction, and enterprise knowledge retrieval with cited sources

 **Common pattern:** Business Goal → Enterprise Data → Tools / APIs → Decision / Workflow → Human Approval → Action



What Makes an Enterprise Agent Different?

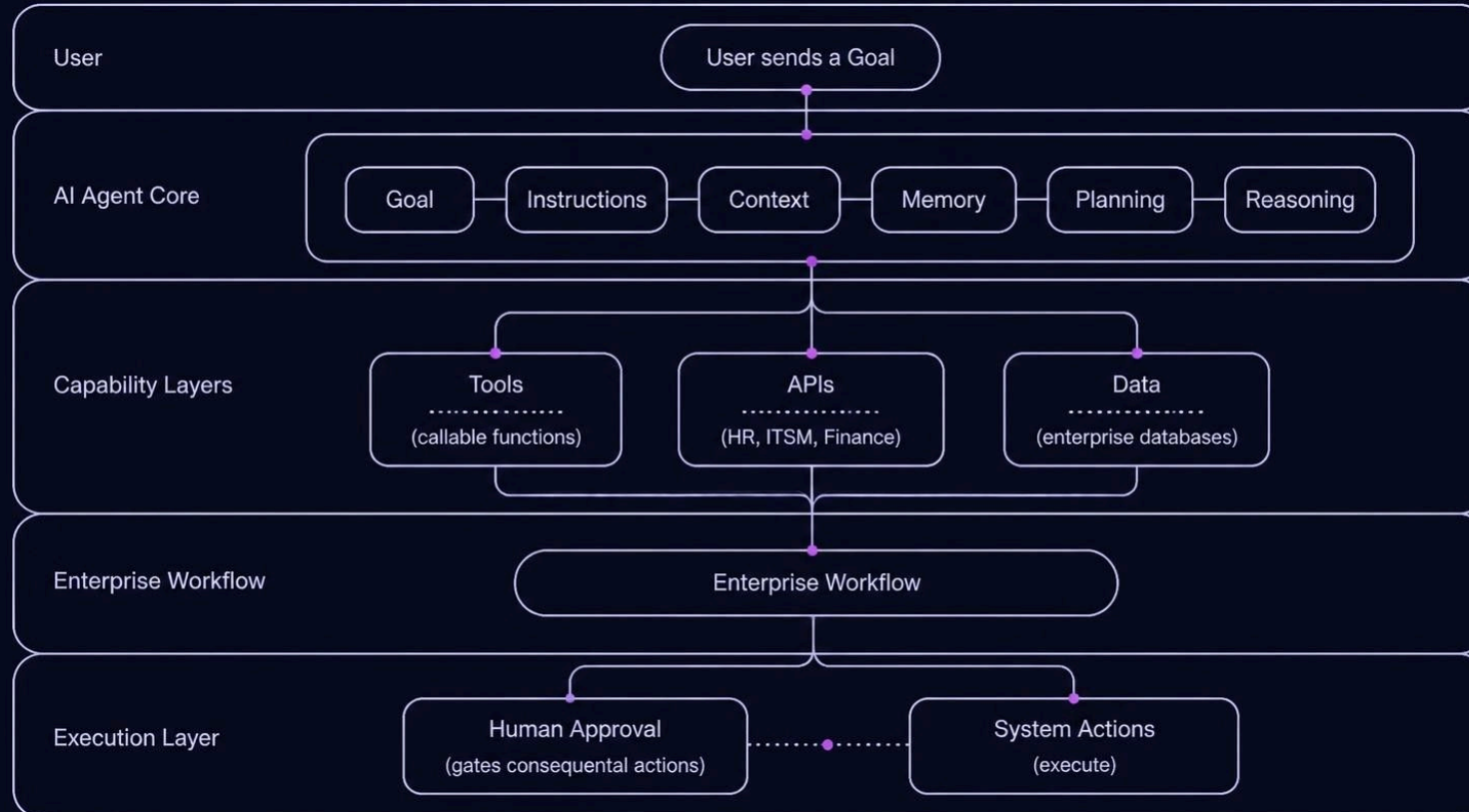
A demo agent running against a public API is not an enterprise agent. A useful enterprise agent requires intelligence, capability, *and* control — all three, deliberately designed.

Capability	Why It Matters in Enterprise
Planning	Break complex, multi-step goals into executable sequences
Tool Usage	Interact with HR, ITSM, finance, and other enterprise APIs
Memory	Maintain relevant context across long-running workflows
Validation	Prevent incorrect or policy-violating actions before execution
Guardrails	Constrain the scope of autonomous action to approved domains
Human Approval	Control consequential, irreversible, or high-risk actions
Observability	Audit and understand every decision the agent made
Error Handling	Recover gracefully from tool failures and unexpected states

Enterprise Agent = Intelligence + Capability + Control

The Architecture We Will Build

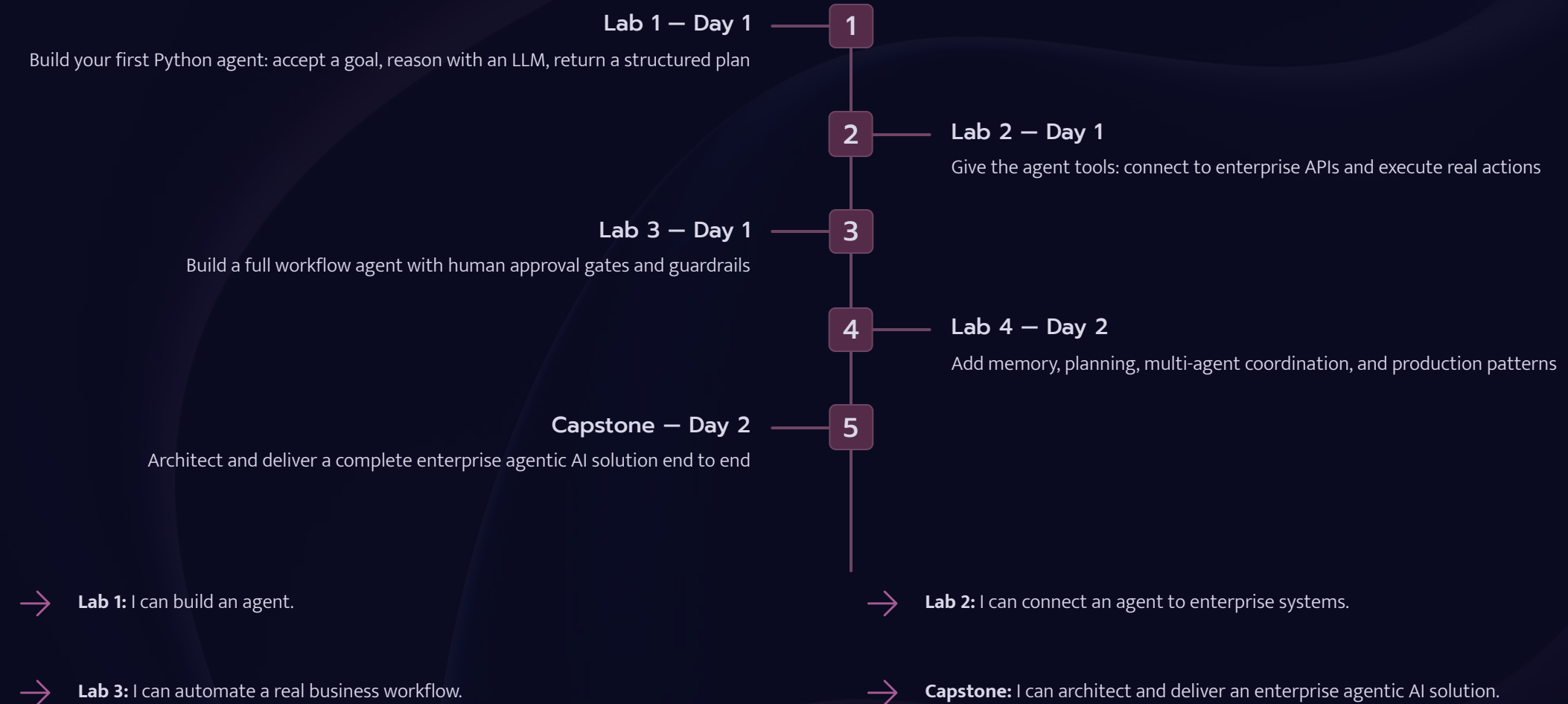
Every component below will be implemented in Python from scratch — no LangGraph, no CrewAI, no AutoGen. This is the architecture that connects user intent all the way through to controlled enterprise action.

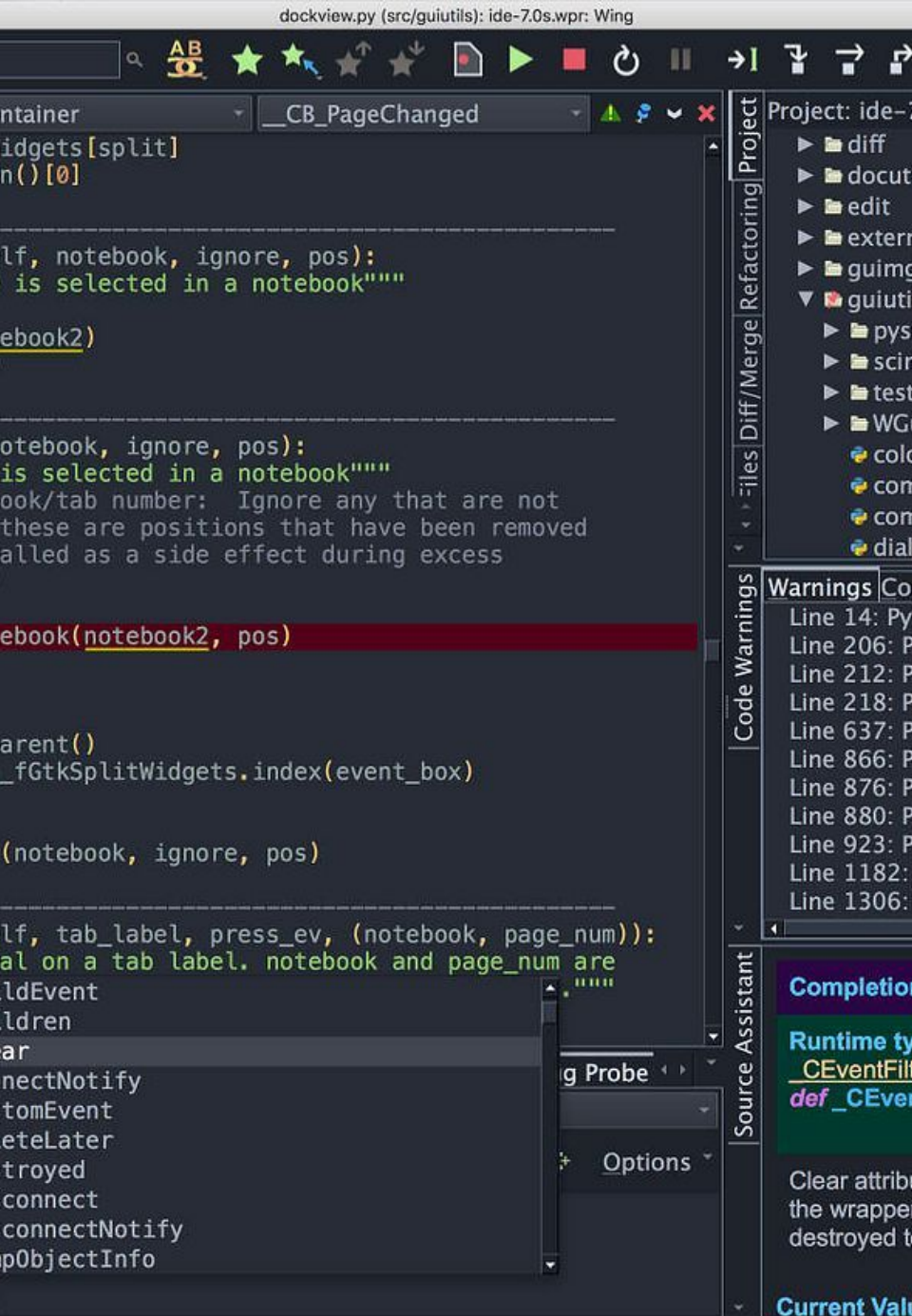


📄 **Framework-independent, Python-first.** The client specifically requested this approach so engineers understand what frameworks abstract away — and can make informed architectural decisions.

Our Learning Journey

The two days are structured as a deliberate progression — each lab builds on the last, culminating in a full enterprise agentic AI solution built entirely in Python.





Now Let's Build One

Enough theory.

Let's build.

The smallest possible agent. A real Python program. A real goal. A real structured output.

The Goal

"Prepare a plan for onboarding a new software engineer."

The Agent

Accepts the goal → sends it to the LLM → receives a structured plan → returns it to the user

The Output

1. Define goal · 2. Prepare workspace · 3. Set up access · 4. Assign mentor · 5. Schedule meetings

📄 **Lab 1 — Build Your First Python AI Agent.** A simple reasoning agent that accepts a goal, asks the LLM to plan, and returns the structured plan. No frameworks. Pure Python.

Key Takeaways

Before we open Lab 1, anchor these five ideas. They are the conceptual foundation for everything that follows.

1 Agentic ≠ Generative

Access to an LLM does not make a system agentic. Agents pursue goals; generative AI produces content.

2 The Loop Is Everything

Think → Plan → Act → Observe → Repeat. Every lab is an implementation of this loop at increasing sophistication.

3 The LLM Is One Component

Intelligence + Tools + State + Instructions + Control Logic = Agent. Remove any one and the system is incomplete.

4 Autonomy Must Be Bounded

Enterprise agents operate within explicit policies, guardrails, and human approval gates — by design.

5 Python First, No Frameworks

We build everything from scratch so you understand what the frameworks abstract — and can make your own architectural choices.